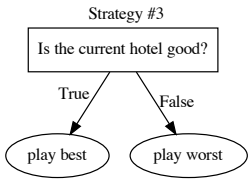
Human Choice Prediction in Non-Cooperative Games: Simulation-based Off-Policy Evaluation - Appendixes

Anonymous Author(s)
Affiliation
Address
email

A Game Strategies

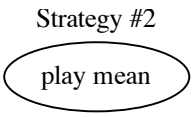
In this appendix, we present the expert strategies of groups $E_{\mathcal{A}}$ and $E_{\mathcal{B}}$ of our game. The formal mathematical notations of the tree node conditions are provided in Table 1 of page 4 in the main paper.

Figure 1: The strategies of the $E_{\mathcal{A}}$ experts.

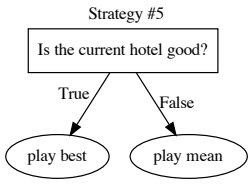


(a) 1st expert's strategy

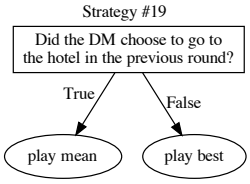
(b) 2nd expert's strategy



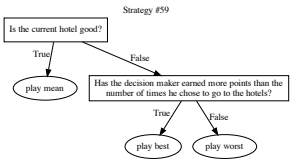
(c) 3rd expert's strategy



(d) 4th expert's strategy



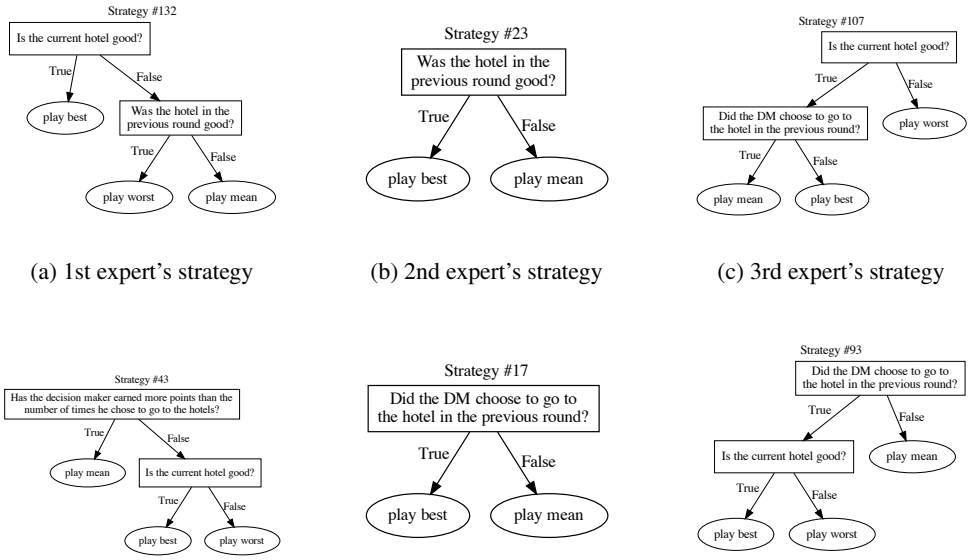
(e) 5th expert's strategy



(f) 6th expert's strategy

Submitted to 37th Conference on Neural Information Processing Systems (NeurIPS 2023). Do not distribute.

Figure 2: The strategies of the $E_{\mathcal{B}}$ experts.



(a) 1st expert's strategy (b) 2nd expert's strategy (c) 3rd expert's strategy

(d) 4th expert's strategy (e) 5th expert's strategy (f) 6th expert's strategy

B Data Collection

B.1 Hotels Dataset

As described in Section 2 of the main paper, our dataset encompassed seven reviews for each of a total of 1068 hotels. The complete dataset can be found in the 'data/game_reviews' directory of the code folder attached to our supplementary material. Figure 3 provides a representative review example.

Figure 3: A sample review from our hotel review dataset.

B.2 Instructions

The following quote contains the instructions given to players in the app stores.

> Are you the vacation planner at your house? Think you always know how to choose the best hotel? Start to plan your 10-days trip with our travel agents. Just remember - they don't always want the best for you, and might have their own strategy to make you book the hotel they try to promote!
>
> Travel or Trouble is a strategy game in which you will try to outsmart our traveling agents and plan the perfect vacation for you.
>
> Each game consists of 10 rounds, in each round, one of our traveling agents will introduce you with a review for a new hotel they think might suit you, and you will have to choose: either book the hotel or stay home.

Only true vacation masters can identify a good hotel based upon one review… are you up to the challenge???

As in life, each vacation can turn out to be a great success or a huge disappointment.

Once you made your choice, you will see the results for the vacation in question: was it good or bad?

Based upon the hotel's average rating (to which only the expert is exposed, and is based on multiple reviews for each hotel), a lottery will determine the outcome of the vacation.

Collect points either by choosing a hotel that turned good or by avoiding bad ones.

Remember - the travel agent is rewarded each time you choose a hotel, regardless of the outcome!

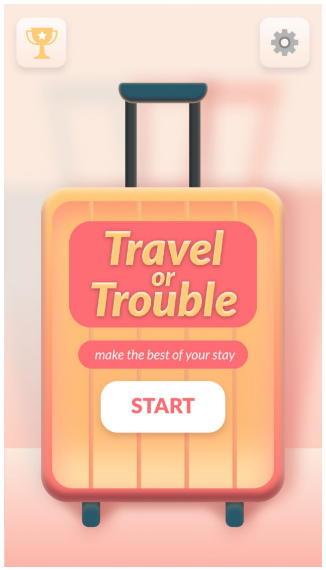
At each game, you will meet a different agent, with a different skill of persuasion.

Try to discover each of our agents' strategies to persuade you, and take the right decision every round.

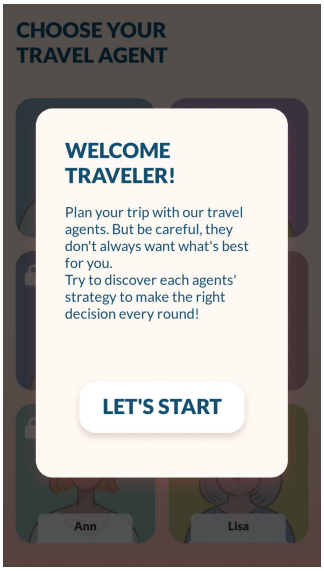
Advance through the world of traveling by earning achievements on your way to becoming the true vacation master.

B.3 App Screenshots

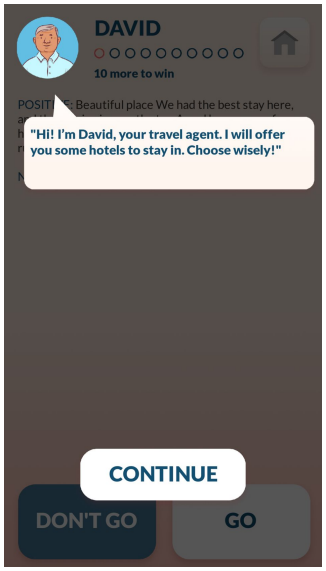
In this appendix, we provide several screenshots from our data collection application.



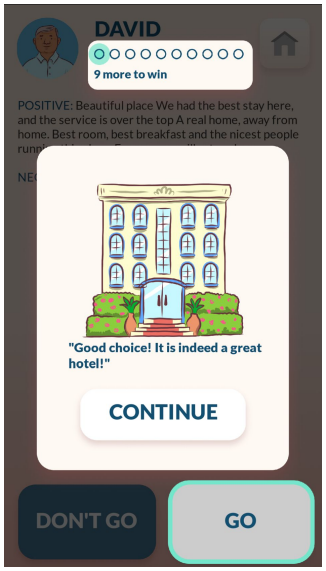
(a) Home Screen



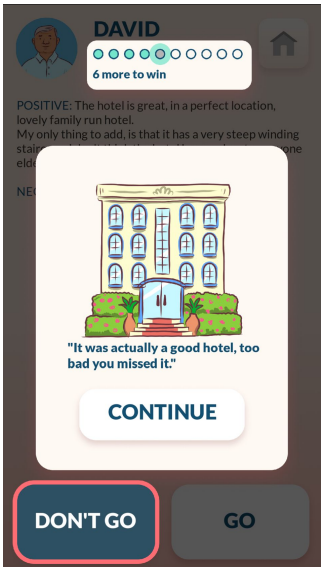
(b) Instructions



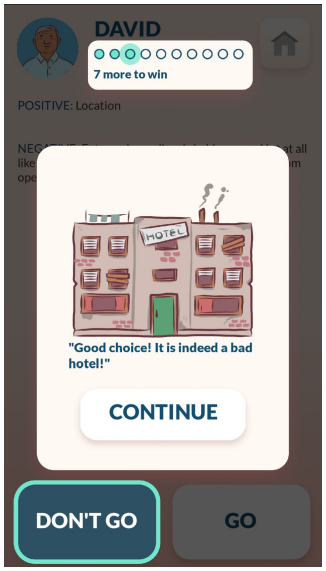
(c) An Agent introduces himself



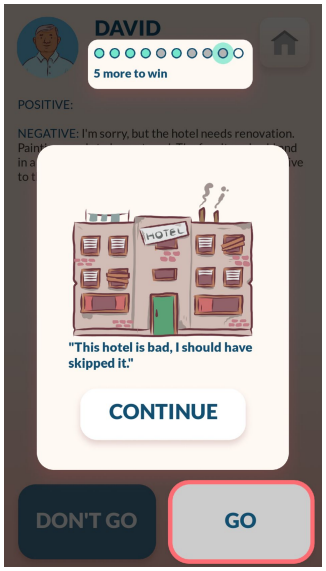
(d) Positive feedback after going to a good hotel



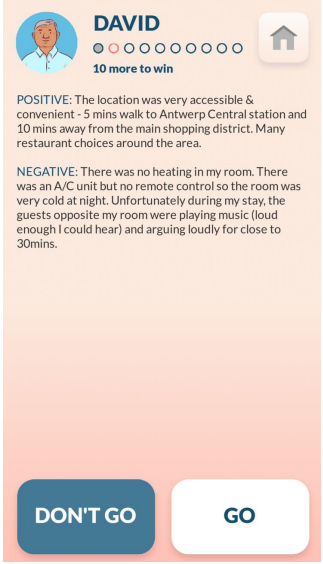
(e) Negative feedback after not going to a good hotel



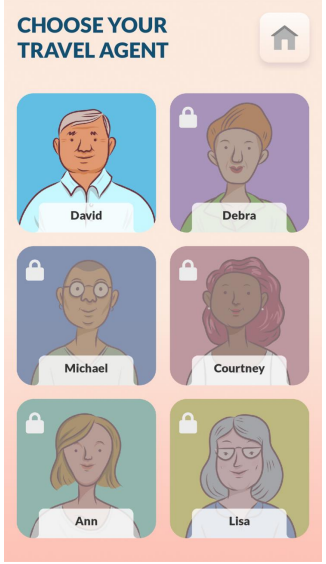
(f) Positive feedback after not going to a bad hotel



(g) Negative feedback after going to a bad hotel



(h) A signal sent by the agent



(i) Agent selection screen

B.4 Information about the human players

As section 3 of the article describes, the app has been available on Google Play and Apple App Store for several months. To attract participants, we also published the app on social media. To increase participation and game completing (playing until defeating all six experts), in some publications we offered participation in a $100 lottery for players who completed the game. In addition, we offered students in an academic course to play the game and complete it in exchange for 0.5 points in the course grade.

C Simulation Pseudo Code

Algorithm 1 describes our simulation. Python code which creates a CSV file with simulation data is provided in the submitted code files.

Algorithm 1 The DM-Expert Simulation

Require: the functions $generate_nature_vector()$, $play_round(user, bot)$, $get_random_bots()$
  procedure $update_temperament(user)$
    for i in $\{1..3\}$ **do**
      $\gamma_i \leftarrow Uni(-\frac{\eta}{10}, \frac{9\eta}{10})$
      $user.temperament_vector[i] \leftarrow (1 - \gamma_i) \cdot user.temperament_vector[i]$
    end for
    if $\sum_{1 \leq i \leq 3} user.temperament_vector[i] > 1$ **then**
      $normalization_factor \leftarrow \sum_{1 \leq i \leq 3} user.temperament_vector[i]$
      for i in $\{1..3\}$ **do**
        $user.temperament_vector[i] \leftarrow \frac{user.temperament_vector[i]}{normalization_factor}$
      end for
    end if
    $user.temperament_vector[0] \leftarrow 1 - \sum_{1 \leq i \leq 3} user.temperament_vector[i]$
  end procedure
  procedure $generate_DM_data(user)$
    $user.nature_vector \leftarrow generate_nature_vector()$
    $user.rivals \leftarrow get_random_bots()$
    for bot in $user.rivals$ **do**
      $user.temperament_vector \leftarrow (0, user.nature_vector)$
      $user.game_points = 0$
      while $user.game_points < 9$ **do**
        $user.temperament_vector \leftarrow (0, user.nature_vector)$
        for round in $\{1..10\}$ **do**
          $play_round(user, bot)$
          $update_temperament(user)$
        end for
      end while
    end for
  end procedure

D Training with simulation Data

Further to Section 4 of the paper, Algorithm 2 describes how we trained neural network models with both interaction and simulation data. To train the XGBoost model, we simply combine the interaction and simulation data to form our training set.

Algorithm 2 Training Loop

Require: interaction_train_dataset
  `pre_training_DS = Create_Simulation_Dataset(n_players =` S_{pt}`)`
  `model.train(dataset = initialization_DS)` ▷ Training with pre-training simulation
  for $epoch \leftarrow 1$ `to` N_{epochs} **do**
    `epoc_DS = Create_Simulation_Dataset(n_players =` S_e`)`
    `model.train(dataset = online_DS)` ▷ Training with epoch simulation
    `model.train(dataset = interaction_train_dataset)`
  end for

E The Input of the Models

All the models in our experiments utilize the same feature representation. Particularly, each DM-bot interaction round is represented using features relating to both the hotel review shared with the DM, and the strategic situation under which the decision was made. Table 1 illustrates the set of binary Hand Crafted Features (HCF), a subset of the feature set originally proposed by Apel et al. (2020), that are used to represent a review. In addition, the table also presents the features we use in order to represent the strategic context of the decision.

Table 1: The text-based and strategic features utilized in our work. All features are binary except for the two counting features (denoted with #).

Features of the review		
Category	**Feature Description**	
Positive Topics	Does the positive part of the reviews provide info. about t?	$t \in$ {Facilities, Price, Design, Location, Room, Staff, View, Transportation, Sanitary Facilities}
Positive Part Properties	Is the positive part empty? Is there a positive summary sentence? Number of characters in range r? Word from group #g[a] in review?	 $r \in$ {[0,99], [100,199], [200,∞)} $g \in [1, 2, 3]$
Negative Topics	Does the negative part of the reviews provide info. about t?	$t \in$ {Price, Staff, Sanitary Facilities, Room, Food, Location, Facilities, Air}
Negative Part Properties	Is the negative part empty? Is there a negative summary sentence? Number of characters in range r? Word from group #g[a] in review?	 $r \in$ {[0,99], [100,199], [200,∞)} $g \in [1, 2, 3]$
Overall Review Properties	Is the ratio between the length of the positive part and the negative part[b] in the range r?	$r \in$ {[0, 0.7], (0.7, 4), [4, ∞)}
Features of the situation		
Category	**Feature Description**	
Strategies Features	Is the previous player action a? Is the previous hotel quality q? # of points DM's earned so far. # of rounds DM's played so far. Points b than rounds played?	$a \in$ {go, not go} $q \in$ {good, not good} $b \in$ {bigger, not bigger}
Reaction time[c]	DM Reaction time in range r seconds?	$r \in$ {[0, 0.5), [0.5, 1), [1, 2), [2, 3), [3, 4), [4, 6.5), [6.5, 12), [12, 20), [20, ∞)}

[a] As described by Apel et al. (2020).
[b] In terms of the number of characters.
[c] Note that the value of this group of features will be 0 for a simulated DM.

F Experiments

F.1 Hyper-parameter Tuning

Model Architecture Selection To identify the most suitable model architecture, we partitioned the DM group that interacts with experts from $E_{\mathcal{A}}$ into two separate groups. Specifically, we randomly selected 80% of the DMs to serve as the training group, and the remaining 20% as the validation group. During the training process, we exclusively utilized the interactions between the training group and the first four of the six agents from $E_{\mathcal{A}}$ as the training data. All the interactions of the

validation group DMs, as well as the interactions between the training group DMs and the last two experts from $E_{\mathcal{A}}$, were employed to assess the model's performance during validation.

To select the hyperparameters for LSTM and LSTM+S, we performed grid search on the following collection of parameters: hidden size $\in$ [32, 64, 128], learning rate $\in$ [$2e^{-4}$, $4e^{-4}$, $8e^{-4}$] number of layers $\in$ [2, 4, 6] $S_{pt} \in [0, 10,000]$, $S_e \in [0, 100]$.

To select the hyperparameters for Transformer and Transformer+S, we performed grid search on the following collection of parameters: hidden size $\in$ [64, 128], learning rate $\in$ [$2e^{-5}$, $8e^{-5}$, $2e^{-4}$] number of layers $\in$ [2, 4], number of heads $\in [2, 4]$, $S_{pt} \in [0, 10,000, 20,000]$, $S_e \in [0, 100, 200]$.

We select the run which achieves the best results while using $S_{pt} = 0, S_e = 0$ to be the no-simulation version of a given model.

In all the experiments we conducted we used seed values $\in [23, 24, 35]$ to reduce noise in architecture selection. The prediction we reported is the average prediction obtained for the sample for the three seed values.

The best results were obtained for hidden size = 128, number of layers = 4, and learning rate = $4e^{-4}$ for the LSTM model, with and without the simulation data. For the version with simulated data, the best results were obtained for $S_{pt} = 10,000$, and $S_e = 100$. For the Transformer model (with and without simulation), the best results were obtained for hidden size = 128, number of layers = 2, number of heads = 2, and learning rate = $2e^{-4}$. For the version with simulated data, the best results were obtained for $S_{pt} = 20,000$, and $S_e = 0$.

Simulation Parameters We adjust the hyperparameters of the simulation model after selecting the appropriate architecture parameters. We tested the $(0, 0.005, 0.1, 0.02)$ values of the η DM improvement parameter, where 0 indicates no improvement over time. To avoid an infinite game, we set a cap of 100 games per expert. We tested all possible combinations of w_1, w_2, and w_3 taken from the set (0, 1, 3, 5) to determine the nature vector. To ensure the temperament vector remains normalized, we calculated its components as $(p_1, p_2, p_3) = (\frac{w_1}{\sum_{i=1}^{3} w_i}, \frac{w_2}{\sum_{i=1}^{3} w_i}, \frac{w_3}{\sum_{i=1}^{3} w_i})$. Finally, we assessed the effect of human score estimation noise by varying ϵ, the standard deviation of the normal distribution we used to generate noise, with the values (0.2, 0.3, 0.4).

Based on our experimental results, the best simulation hyperparameters were found to be $\eta = 0.01$, $(p_1, p_2, p_3) = (0, \frac{3}{4}, \frac{1}{4})$, and $\epsilon = 0.3$. We employ these values in our subsequent analysis, as presented in the following section.

F.2 Compute Information

The deep learning models were trained on a device with an Intel Core i9-7920X CPU operating at 2.90GHz, which features 24 threads spread across 12 cores. This machine is equipped with two NVIDIA GeForce GPUs, one of which was dedicated to the training task, as it used 707MB for LSTMs models and 739MB for Transformers models, from a total of 11178MB (roughly 11GB) memory. The generous memory and storage resources ensured smooth training of the deep learning model. The training was performed under an operating system running on the x86_64 architecture.

The training time of LSTM+S was 56 minutes. The training time of the Transformer+S was 184 minutes.

G Results for Hard-to-Predict Decisions

In section 6 of the paper, we present the performance of the various models on those cases where the probability a given model assigns to the two possible actions is at least 0.2. In Tables 2, 3, 4, and 5 we report the same analysis where everything is kept fixed except that the hyper-parameter of 0.2 is replaced with other values.

Table 2: Performance of the various models (rows) on those cases where the probability the model in the column title assigns to the two possible actions is at least 0.05.

\ Hard to Accuracy of \	Majorirty	XGBoost	LSTM+S	LSTM	Trans.	all baselines	one baseline
LSTM+S	**75.98%**	**76.65%**	**73.58%**	**76.05%**	**76.79%**	**70.37%**	**79.28%**
LSTM	74.49%	74.77%	71.12%	73.9%	74.87%	68.42%	77.82%
XGBoost+S	74.56%	75.11%	72.28%	74.84%	75.61%	69.59%	78.11%
XGBoost	73.52%	73.74%	71.29%	73.9%	74.36%	67.68%	77.21%
TF+S	73.23%	73.25%	70.2%	72.89%	73.55%	67.14%	76.74%
TF	72.69%	73.27%	69.5%	72.53%	73.23%	66.17%	76.51%
Majority	69.15%	71.4%	68.88%	71.48%	71.06%	65.6%	72.92%
# Decisions	6846	8091	7566	7996	8660	4433	10835
% Decisions	47.94%	56.66%	52.99%	56.0%	60.65%	31.05%	75.88%

Table 3: Performance of the various models (rows) on those cases where the probability the model in the column title assigns to the two possible actions is at least 0.1..

\ Hard to Accuracy of \	Majorirty	XGBoost	LSTM+S	LSTM	Trans.	all baselines	one baseline
LSTM+S	**74.34%**	**74.5%**	**70.04%**	**73.51%**	**73.76%**	**65.46%**	**77.49%**
LSTM	73.19%	72.07%	66.43%	69.93%	70.71%	62.7%	75.69%
XGBoost+S	73.25%	72.58%	68.79%	72.0%	71.72%	64.62%	76.09%
XGBoost	71.84%	70.87%	67.76%	71.01%	70.82%	63.28%	75.16%
TF	71.63%	70.49%	65.36%	68.52%	68.36%	61.64%	74.2%
TF+S	71.47%	70.4%	65.98%	69.43%	69.12%	61.29%	74.56%
Majority	66.92%	68.3%	65.53%	68.33%	66.92%	61.56%	70.57%
# Decisions	5747	6737	6268	6725	7227	3082	9823
% Decisions	40.25%	47.18%	43.9%	47.1%	50.61%	21.58%	68.79%

Table 4: Performance of the various models (rows) on those cases where the probability the model in the column title assigns to the two possible actions is at least 0.15.

\ Hard to Accuracy of \	Majorirty	XGBoost	LSTM+S	LSTM	Trans.	all baselines	one baseline
LSTM+S	**72.15%**	**72.86%**	**64.99%**	**67.93%**	**69.45%**	**61.38%**	**75.15%**
LSTM	70.27%	69.84%	60.55%	63.49%	65.56%	57.95%	73.04%
XGBoost+S	70.62%	70.45%	63.63%	66.64%	67.13%	59.63%	73.69%
XGBoost	69.42%	68.5%	62.83%	65.7%	66.5%	57.84%	72.74%
TF+S	68.76%	67.9%	60.35%	63.65%	64.04%	57.48%	71.86%
TF	68.7%	68.47%	59.29%	62.41%	62.99%	57.82%	71.47%
Majority	64.03%	65.86%	61.71%	63.15%	62.04%	57.03%	67.83%
# Decisions	4691	5608	5012	5252	5744	1903	8762
% Decisions	32.85%	39.27%	35.1%	36.78%	40.23%	13.33%	61.36%

Table 5: Performance of the various models (rows) on those cases where the probability the model in the column title assigns to the two possible actions is at least 0.25.

\ Hard to Accuracy of \	Majorirty	XGBoost	LSTM+S	LSTM	Trans.	all baselines	one baseline
LSTM+S	**70.18%**	**70.25%**	57.56%	**62.91%**	**65.28%**	**59.49%**	**70.67%**
LSTM	67.43%	65.85%	53.72%	56.08%	59.33%	52.21%	67.08%
XGBoost+S	67.41%	66.59%	**58.12%**	62.22%	61.86%	57.37%	68.7%
XGBoost	65.43%	63.1%	57.06%	60.47%	61.24%	51.06%	67.07%
TF+S	66.1%	64.05%	53.78%	58.62%	58.08%	52.73%	66.09%
TF	65.82%	62.78%	52.09%	57.51%	57.05%	55.95%	65.51%
Majority	61.56%	61.07%	56.09%	58.87%	58.02%	54.86%	62.86%
# Decisions	2663	3315	2901	2988	3393	373	6562
% Decisions	18.65%	23.22%	20.32%	20.93%	23.76%	2.61%	45.96%